

RESEARCH ARTICLE

Hybrid Approach to Software Fault Prediction Using Particle Swarm Optimization and Fuzzy Time Series

Umashankar Samal 

Department of Mathematics, GLA University, Uttar Pradesh, India

Correspondence: Umashankar Samal (umashankar.samal249@gmail.com)

Received: 16 December 2024 | **Revised:** 26 February 2025 | **Accepted:** 27 February 2025

Funding: No funding is received for this research work.

Keywords: forecasting | fuzzy time series | particle swarm optimization | software reliability

ABSTRACT

Accurate software fault prediction is a critical aspect of proactive maintenance and overall software reliability enhancement. This paper introduces a model that combines particle swarm optimization (PSO) and fuzzy time series (FTS) to predict the number of faults present in the software. FTS, widely recognized for its forecasting capabilities, undergoes performance optimization in the proposed model through PSO, specifically by optimizing the length of intervals. The performance of the proposed model is evaluated on three real-world software fault datasets sourced from open-source software projects, revealing its superiority over alternative models. The evaluation is substantiated by consistently lower values of root mean absolute error, mean absolute error, and median absolute error. This research significantly contributes to the advancement of software reliability forecasting techniques, providing practitioners with a powerful tool for ensuring software reliability and enabling proactive measures for continuous improvement.

1 | Introduction

In recent decades, rapid advancements in operating systems, applications, and control programs across diverse scientific and engineering fields have underscored the critical need for reliable software systems. The dependence on computer systems and software programs makes it imperative to address the potential breakdowns resulting from software errors, which can have severe consequences. Several real-world incidents highlight the significance of software reliability [1]. In 2015, a software malfunction in the US correctional system led to the premature release of inmates, demonstrating the critical consequences of faulty software [2]. Similarly, the crash of the Airbus A400M in Spain was attributed to software failure, emphasizing the need for robust testing and verification processes in aviation systems [3]. These incidents underscore the crucial role software

reliability plays in ensuring safety and system effectiveness across various industries. As modern technology increasingly relies on software-driven systems, rigorous reliability practices are essential to minimize risks and maintain operational integrity. Consequently, the study of software reliability has become a focal point for researchers and scientists. Software reliability is a multidimensional concept that goes beyond mere functionality. It encompasses the ability of a software system to perform its intended functions without failure, consistently meeting user expectations in diverse operating conditions [4]. Achieving software reliability involves navigating through the process of design, coding, testing, and maintenance. The goal is not only to eliminate existing defects but also to preclude the emergence of new ones. The dynamic nature of software, marked by continuous updates and evolving user requirements, necessitates a proactive approach to reliability management.

Efforts to investigate software reliability have led to the development of quantitative analysis techniques, particularly during the testing phase of the software development process. Software reliability growth models (SRGMs) have played a crucial role in quantifying and improving software reliability. Over the past few decades, numerous SRGMs have been developed and reported in the literature, addressing various aspects of the software development process [5–9]. Many of these models have been built upon the assumption of immediate fault removal and perfect debugging, overlooking the imperfections inherent in the debugging process [10]. To account for these imperfections, alternative SRGMs have been proposed [11, 12]. However, most parametric SRGMs are grounded in assumptions that may not hold true in practical scenarios, leading to over-optimistic estimates of software reliability.

Recognizing the limitations of conventional SRGMs, researchers have explored the applicability of machine learning techniques, forecasting techniques, and so forth, in software reliability modeling [13–17]. Forecasting, a pivotal area of research, has captivated the attention of numerous researchers and scientists, striving to refine the accuracy and reliability of predictions. Amidst various established forecasting models, fuzzy time series (FTS) models have emerged as powerful tools, outperforming traditional methods [18–20]. FTS forecasting involves the application of linguistic mathematical reasoning to model and predict future trends based on a time series of linguistic historical observations. Unlike traditional approaches, FTS models consider uncertainties in observations over time, avoiding restrictive assumptions and minimizing the need for extensive background knowledge.

Fluctuation, a characteristic inherent in time series data, serves as a valuable attribute for analyzing patterns and making predictions. FTS stands out for its descriptive power compared to traditional time series methods, offering semantic meaning for handling uncertain and fluctuating data. The introduction of FTS forecasting by Song and Chissom [21, 22], who applied it to predict student enrollment at Alabama University, marked the inception of a concept that has continuously evolved through substantial contributions from researchers in this field.

Typically, the FTS forecasting model is structured into sequential phases, including defining the universe of discourse (UOD), partitioning the UOD, fuzzifying historical data, establishing fuzzy logical relationships (FLRs) and their grouping, and optionally defuzzification [23–25]. The phase of partitioning the UOD involves the operation of dividing the UOD into unequal or equal partitions. The key hyperparameters influencing the efficiency of FTS forecasting encompass the number of partitions in UOD, length of partition intervals, and the fuzzy order of the model. Existing studies have primarily focused on optimizing one or two hyperparameters of FTS to enhance forecasting results [23, 24].

Cai et al. [26] have emphasized the potential of fuzzy set theory in software reliability modeling, highlighting its capacity to address the uncertainties associated with human behavior and dynamic software environments. Fuzzy set theory accommodates the variability in design skills among different software professionals, as well as changes in design personnel during a project. Furthermore, it acknowledges the fuzziness in software execu-

tion environments, encompassing system software, machines, and platforms.

In the pursuit of optimizing FTS hyperparameters, various optimization techniques have been proposed in the literature [27, 28]. Among these, the particle swarm optimization (PSO) algorithm has garnered significant attention in recent years, particularly for determining the optimum length of intervals in UOD [29]. The utilization of hybrid techniques integrating lessons from existing studies holds the potential to advance the effectiveness of FTS forecasting by optimizing multiple hyperparameters simultaneously.

In this research paper, an FTS forecasting model is introduced for forecasting software faults through the integration of the PSO algorithm for optimized hyperparameters. Specifically, PSO is used to optimize the length of the partitioned interval. The following portions of this paper are organized as follows: Section 2 explores the literature relevant to the proposed model, emphasizing the contributions. Section 3 provides an in-depth exploration of fuzzy set theory, FTS, PSO, and the architecture of the proposed model. In Section 4, the dataset employed and the criteria used for model comparison are detailed. Section 6 discusses the limitations and threats to validity. Section 5 discusses the obtained results. The concluding remarks and insights into future directions are presented in Section 7.

2 | Literature Review

Numerous research studies have explored SRGMs over time, with a focus on estimating software reliability through the analysis of software flow data. It is crucial to recognize that there is not a one-size-fits-all SRGM capable of universally depicting software behavior across all scenarios. Each software project exhibits distinctive characteristics and demands, emphasizing the need for thoughtful assessment and customization of SRGMs to effectively capture the unique dynamics.

The Goel–Okumoto model, referred to as the GO model, stands out as a prominent model compliant with the nonhomogeneous Poisson process (NHPP) [30]. This model's underlying assumption is that the frequency of fault detection should decline linearly with calendar time as the number of undetected faults decreases over time. Yamada et al. [31] presented a delayed S-shaped model that took into account the lag period between fault identification and elimination. An S-shaped fault detection rate (FDR) was included in this model to better describe how the rate of fault detection changes over time.

Sahu and Srivastava [32] conducted research focused on reviewing established methodologies for reliability prediction, offering a critical assessment of successful studies in this domain. The paper systematically discusses the SRGMs in the context of software development, highlighting challenges and key considerations associated with the estimation of reliability. In a noteworthy investigation, Haque and Ahmad [33] compiled an extensive list of these models. Nevertheless, it is crucial to recognize that certain models have limitations, such as the inclusion of unrealistic assumptions, fault adjustment without introducing new faults, or presumptions of independence between failures

and so forth [10, 30–32]. To tackle these challenges, time-series forecasting has gained prominence as a crucial task in the development of reliable systems. This approach proves especially valuable when there is limited knowledge available about the data generation process.

In recent decades, the literature has witnessed the emergence of a varied spectrum of FTS forecasting models. Bose and Mali [25] conducted a comprehensive review of FTS forecasting, organizing the stages of the process into distinct categories: (1) defining UOD, (2) partition of UOD, (3) fuzzification, (4) establishing FLRs, and (5) defuzzification if needed. This categorization was further structured into two primary phases: (a) data partitioning and (b) prediction Phase, providing a systematic framework for understanding the evolution and intricacies of FTS forecasting methodologies.

Pattanayak et al. [34] introduced an FTS forecasting model that utilizes fuzzy cognitive maps (FCM) to determine non-uniform interval lengths. Support vector machines (SVM) are employed to ascertain FLRs, while autocorrelation and partial autocorrelation functions are leveraged to determine the model order. In a related study, Panigrahi and Behera [35] addressed two crucial aspects of higher-order FTS forecasting. They tackled the optimization of interval lengths in the UOD using a modified average-based method and employed machine learning techniques, specifically long short-term memory (LSTM), deep belief network (DBN), and SVM, for modeling FLRs.

Cai et al. [26] explored the application of fuzzy set theory in software reliability modeling, emphasizing its advantages in being assumption-free and more flexible. They noted that the entire software development process is inherently influenced by human factors, and human behavior, being inherently fuzzy, is effectively captured by fuzzy set theory. Moreover, the execution of software is contingent on the environment, encompassing system software, machines, and platforms, which introduces a fuzzy dimension. Software is designed with human involvement, and design defects primarily contribute to software faults. Given the variability in design skills among software professionals and potential changes in design personnel during a project, the fuzzy nature of human involvement is accentuated. Chatterjee et al. [36] proposed two software reliability models: a simple FTS model with time between failures (TBFs) as a fuzzy linguistic variable, and a more complex two-factor higher-order FTS model.

Junhong et al. [37] proposed a nonlinear software reliability model utilizing time series and provided associated algorithms. In this approach, fuzzy theory is incorporated for software reliability assessment. By integrating fuzziness and randomness through fuzzy theory, the evaluation method becomes more closely aligned with real system characteristics, significantly enhancing the accuracy of software reliability assessments. Aljahdali and Sheta [38] proposed a fuzzy software reliability model comprises a set of interconnected linear sub-models combined through fuzzy membership functions to represent the overall fuzzy model. The authors investigated the application of fuzzy logic for forecasting faults in the software testing phase, leveraging historical software faults data. Zhao et al. [39] employed phase space reconstruction technology and the fuzzy wavelet neural network (FWNN) to introduce an SRGM known as the FWNN-based SRGM. Unlike

the challenges faced by the back-propagation neural network (BPNN) in determining network architecture, which involves predefining parameters like the number of neurons and layers, the authors aimed to address this issue by leveraging phase space reconstruction technology along with FWNN to enhance prediction accuracy. This approach not only overcame the architectural difficulties associated with BPNN but also facilitated the determination of delay time and embedding dimension through phase space reconstruction. Chatterjee et al. [40] proposed a stochastic differential equation SRGM, considering fault severity and priority in defect resolution. The authors addressed optimal software release timing with high reliability and minimal development costs, incorporating randomized budgets represented by interval type-2 fuzzy numbers to capture dynamic specification changes. Kumar [41] proposed a neuro-fuzzy hybrid method, combining self-organized map (SOM) and FTS for software reliability prediction. The approach integrates the supervised clustering algorithm SOM with FTS forecasting, demonstrated through an experimental study on a software failure dataset.

Contribution

The efficiency of FTS forecasting relies on critical hyperparameters, including the number of partitions in the UOD, length of partition intervals, and fuzzy order. In the field of software reliability, existing research predominantly focuses on elementary FTS models, even though FTS forecasting techniques have demonstrated improved prediction results across diverse domains. However, there is a notable scarcity of works within software reliability that specifically address the optimization of FTS hyperparameters. Building upon insights gleaned from prior studies, the contribution involves leveraging hybrid techniques to optimize various FTS hyperparameters. Specifically, an FTS forecasting model with hyperparameters fine-tuned using PSO is proposed, with a particular emphasis on optimizing the length of the partition interval to enhance prediction accuracy in software reliability.

3 | Background

This section outlines the fundamental principles of FTS and PSO applied in the current investigation and also, discusses the proposed model architecture.

Fuzzy Sets

Fuzziness, in essence, refers to “vagueness.” Fuzzy set theory, introduced by Zadeh [42], serves as a powerful mathematical tool to address uncertainty stemming from vagueness. The concept of a fuzzy set allows for a variable degree of membership (ranging between 0 and 1) for elements within a set [43].

For a fuzzy set A defined on the universal set U , it is expressed as:

$$A = \frac{f_A(u_1)}{u_1} + \frac{f_A(u_2)}{u_2} + \dots + \frac{f_A(u_n)}{u_n}. \quad (1)$$

Here, f_A represents the membership function of A , where $f_A : U \rightarrow [0, 1]$, and $f_A(u_i)$ denotes the membership degree of element u_i in the fuzzy set A for $1 \leq i \leq n$. Unlike classical sets where an element either belongs or does not belong to a set (i.e., membership values are restricted to $\{0, 1\}$), fuzzy sets allow elements to have varying degrees of membership. This property is crucial in modeling real-world phenomena where precise categorization is difficult. For instance, concepts such as “tall people,” “warm weather,” or “high reliability” cannot be easily defined with strict boundaries but can be better represented using fuzzy sets.

Membership functions in fuzzy sets can take various forms depending on the application. The triangular membership function is defined by a triplet (a, b, c) , where a and c are the lower and upper bounds, and b is the peak. The trapezoidal membership function is similar to the triangular function but allows for a plateau at the peak, making it more suitable for scenarios where a certain range of values has equal significance. The Gaussian membership function, on the other hand, employs a bell-shaped curve and is commonly used for smooth transitions due to its continuous and differentiable nature. The flexibility in defining membership functions makes fuzzy sets particularly useful in decision-making, control systems, pattern recognition, and various artificial intelligence applications [44].

Fuzzy Time Series

Consider $Y(t)$, where $t = 0, 1, 2, \dots$ as a subset of $\mathbb{R}$, serving as the UOD on which fuzzy sets $f_i(t)$, where $t = 0, 1, 2, \dots$, are defined. Let $F(t)$ comprise $f_1(t), f_2(t), \dots$. In this context, $F(t)$ is referred to as a FTS defined on $Y(t)$. Here, $F(t)$ is considered a linguistic variable, and $f_i(t)$ represents potential linguistic values of $F(t)$.

If a fuzzy relationship $R(t-1, t)$ exists, such that $F(t) = F(t-1) \times R(t-1, t)$, where $\times$ represents an operator, then $F(t)$ is considered to be caused by $F(t-1)$. This causal relationship between $F(t)$ and $F(t-1)$ is denoted by $F(t) \rightarrow F(t-1)$. If $F(t)$ is influenced by $F(t-1), F(t-2), \dots, F(t-m)$, then this fuzzy relationship is denoted as $F(t-m), \dots, F(t-2), F(t-1) \rightarrow F(t)$, and is referred to as an m -order FTS. A detailed description for the steps involved in the FTS is given in the Figure 1. FTS is particularly useful in forecasting scenarios where data is inherently uncertain or exhibits imprecise patterns. Applications of FTS include stock market prediction, weather forecasting, traffic flow estimation, and software reliability analysis. By integrating fuzzy logic into time series modeling, FTS allows for more adaptable and interpretable forecasting models that better capture the underlying uncertainties of real-world data.

Particle Swarm Optimization

PSO stands as a bio-inspired algorithm renowned for its simplicity in navigating solution spaces to identify optimal solutions. Distinguished from other optimization algorithms, PSO relies solely on the objective function, operating independently of gradients or differential forms of the objective. This characteristic renders it highly versatile, particularly in scenarios where such information is challenging to obtain or compute. Notably, PSO

exhibits simplicity in terms of hyperparameters, requiring very few to facilitate its application. The choice of PSO over other metaheuristic algorithms, such as genetic algorithm (GA) or differential evolution (DE), is justified based on several factors. PSO is computationally efficient due to its simple update equations, requiring fewer control parameters than GA and DE, making it easier to implement and tune [45, 46]. Unlike GA, which involves complex crossover and mutation operations, PSO relies on velocity and position updates, leading to faster convergence [47]. Furthermore, PSO maintains a balance between exploration and exploitation, reducing the likelihood of premature convergence, which is a common issue in DE. Additionally, PSO has been successfully applied in various optimization problems, including time series forecasting and hyperparameter tuning, demonstrating its robustness in handling dynamic search spaces. Given these advantages, PSO is well-suited for optimizing the parameters of the FTS model, enhancing its predictive performance.

In PSO, a swarm of particles traverses an n -dimensional search space, with each particle representing a potential solution. The position of the k^{th} particle is denoted by $X_k = [x_k^1, x_k^2, \dots, x_k^n]$, and its velocity by $V_k = [v_k^1, v_k^2, \dots, v_k^n]$, where P signifies the number of particles in the swarm. Particles begin with random values within the search space. Each particle maintains its personal best position P_{best_k} , and the global best position P_{gbest} is recorded. Particle velocity and position are updated using the Equations (2) and (3).

$$V_k^{t+1} = w^t * V_k^t + C_1 * \text{Rand}() * (P_{\text{best}_k} - X_k^t) + C_2 * \text{Rand}() * (P_{\text{gbest}} - X_k^t) \quad (2)$$

$$X_k^{t+1} = X_k^t + V_k^{t+1} \quad (3)$$

The inertia weight w_t is adjusted using the equation

$$w_t = w_{\text{max}} - \frac{t * (w_{\text{max}} - w_{\text{min}})}{t_{\text{max}}}, \quad (4)$$

where w_{max} and w_{min} are predefined values, t is the current iteration, and t_{max} is the maximum iteration count.

Particle velocity V_k^t is constrained to the range $[-V_{\text{max}}, V_{\text{max}}]$, while particle position X_k^t is confined within the UOD range $[U_{\text{min}}, U_{\text{max}}]$. Particle positions are initialized using the equation

$$x = U_{\text{min}} + \text{Rand}() * (U_{\text{max}} - U_{\text{min}}). \quad (5)$$

3.1 | Proposed Methodology

The proposed PSO based FTS model for software fault prediction integrates PSO into the training phase to optimize both forecasting rules and parameters. The training framework is systematically governed by three key algorithms. First, Algorithm 1 initiates the iterative optimization process, initializing parameters and conducting epochs where particles, representing potential solutions, are assessed within the UOD. Local and global best particles undergo continual updates, refining particle movements through adjustments to swarm parameters.

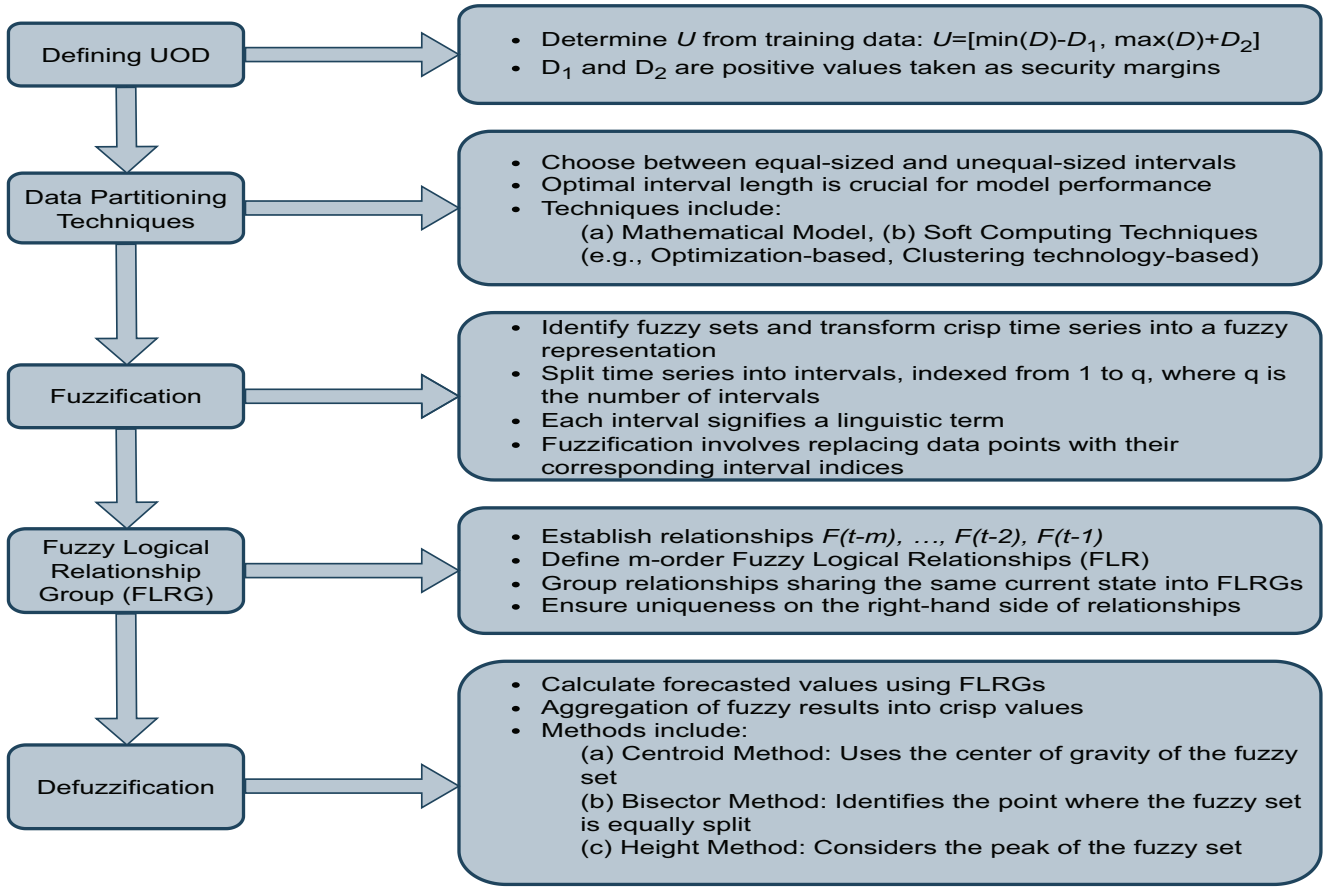


FIGURE 1 | Steps involved in FTS.

Algorithm 2, the update function, dynamically adapts particle velocities and positions based on crucial factors, incorporating bounds checking to ensure adherence to specified limits. Lastly, Algorithm 3, the evaluateUOD function, is central to UOD assessment during training. It constructs intervals, evaluates fault memberships, and computes pivotal metrics such as root mean squared error (RMSE) and mean absolute error (MAE). Collectively, these algorithms underpin the systematic training of the hybrid PSO-FTS model, facilitating the optimization of particle movements and the shaping of UOD for enhanced software fault prediction accuracy.

4 | Experimental Setting

4.1 | PSO Parameter Selection and Initialization

One of the challenges in PSO is that its performance is highly sensitive to the initial placement and distribution of particles, which can impact the consistency and reliability of the optimization process. To address this issue, Clerc and Kennedy [48] introduced a constriction coefficient to enhance convergence stability and prevent premature divergence. The velocity update equation in PSO, incorporating this constriction factor, is given by:

$$v(t+1) = k \cdot [v(t) + c_1 r_1 (p(t) - x(t)) + c_2 r_2 (g(t) - x(t))], \quad (6)$$

$$x(t+1) = x(t) + v(t+1), \quad (7)$$

where $v(t)$ and $v(t+1)$ denote the velocity at time t and $t+1$, respectively, and $x(t)$ represents the position of a particle. The terms $p(t)$ and $g(t)$ correspond to the personal and global best positions found so far, while c_1 and c_2 are acceleration coefficients that control the balance between exploration and exploitation. The variables r_1 and r_2 are random values uniformly distributed in the range $[0,1]$. The constriction coefficient k is given by:

$$k = \frac{2}{|2 - \phi - \sqrt{\phi^2 - 4\phi}|}, \quad (8)$$

where $\phi = c_1 + c_2$. Clerc and Kennedy [48] recommend setting $c_1 = c_2 = 2.05$, which results in $\phi = 4.1$ and yields a constriction coefficient of approximately $k \approx 0.729$. This value ensures that the velocity updates remain stable and prevent divergence. Additionally, the inertia weight is typically set to $w \approx 0.7298$ to further enhance convergence properties. By incorporating this approach, the PSO-driven optimization in our model remains stable across different initial conditions, thereby improving the reliability of fault prediction using FTS.

4.2 | Dataset

This study conducts a comparative evaluation of the proposed model using three distinct datasets for fault detection, previously utilized by Wu and Huang [49]. The first dataset (DS-1), detailed in Table A1, contains 8780 recorded faults over 208 months and is

ALGORITHM 1 | Training the Model.

```

1: procedure TRAINMODEL (series, faults, epoch, option)
2:   Initialize all necessary parameters
3:   while epoch > 0 do
4:     for  $i \leftarrow 0$  to  $N - 1$  do
5:       Compute necessary values using EVALUATEUOD()
6:       if  $L_{\text{best}} \text{RMSE}[i] > \text{RMSE}$  then
7:         Update  $L_{\text{best}}$  particles and related values
8:       end if
9:       if  $G_{\text{best}} \text{RMSE} > \text{RMSE}$  then
10:        Update  $G_{\text{best}}$  particle and related values
11:      end if
12:    end for
13:    Update  $w$  and related parameters
14:    for  $i \leftarrow 0$  to  $N - 1$  do
15:      Update particles and velocities using UPDATE()
16:    end for
17:    epoch  $\leftarrow$  epoch  $- 1$ 
18:  end while
19:  return  $G_{\text{best}}$  particle,  $G_{\text{best}}$  RMSE,  $G_{\text{best}}$  MAE, GETMDAE(FT)
20: end procedure

```

derived from the JDT component of Eclipse version 2.0, an open-source software project. The second dataset (DS-2), presented in Table A2, includes 14,608 faults recorded over 188 months from the platform component of Eclipse version 3.0. The third dataset (DS-3), shown in Table A3, consists of 2251 faults recorded over 193 months and originates from the networking component of the Linux operating system.

4.3 | Performance Measures

To assess the accuracy of the proposed model, commonly used metrics such as the MAE, RMSE, and median absolute error (MdAE) are employed which are defined by Equations (9), (10), and (11), respectively. The MAE is a metric that quantifies the average absolute difference between the actual and predicted values in a dataset. It yields a non-negative value, where a lower MAE indicates higher accuracy. RMSE measures the average magnitude of the differences between predicted values and the actual values. It is a way to quantify the spread or dispersion of prediction errors. MdAE is a metric used to evaluate the accuracy of a predictive model, particularly in the context of forecasting or regression analysis.

$$\text{MAE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}. \quad (9)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (10)$$

ALGORITHM 2 | Update Function.

```

1: function update ( $k, w, C_1, C_2, G_{\text{best\_particle}},$   

    $L_{\text{best\_particle}}, \text{particle}, \text{velocity}, \text{lower\_bound},$   

    $\text{upper\_bound}$ )
2:    $\text{updated\_velocity} \leftarrow k \times (w \times \text{velocity} + C_1 \times$   

      $\text{random}() \times (L_{\text{best\_particle}} - \text{particle}) + C_2 \times$   

      $\text{random}() \times (G_{\text{best\_particle}} - \text{particle}))$ 
3:    $\text{updated\_particle} \leftarrow \text{particle} + \text{updated\_velocity}$ 
4:   Create a dataframe from  $\text{updated\_particle}$  and  

      $\text{updated\_velocity}$ 
5:   function check ( $\text{position}$ )
6:     if  $\text{position} < \text{lower\_bound}$  then
7:       return  $\text{lower\_bound}$ 
8:     else if  $\text{position} > \text{upper\_bound}$  then
9:       return  $\text{upper\_bound}$ 
10:    else
11:      return  $\text{position}$ 
12:    end if
13:  end Function
14:  Map check ( $\text{position}$ ) to the “particle” column in the  

   dataframe
15:  Sort the dataframe based on the “particle” column in  

   ascending order
16:  Update  $\text{updated\_particle}$  with the sorted “particle”  

   column from the dataframe
17:  Update  $\text{updated\_velocity}$  with the sorted “velocity”  

   column from the dataframe
18:  return  $\text{updated\_particle}, \text{updated\_velocity}$ 
19: end Function

```

$$\text{MdAE} = \text{median}(|\text{predicted} - \text{actual}|). \quad (11)$$

5 | Results and Discussion

In this section, a comprehensive comparison is conducted of the proposed PSO-FTS model with several established models, including FTS, artificial neural network (ANN), support vector machine (SVM), k-nearest neighbors (KNN), decision tree (DT), and random forest (RF). The assessment criteria involved key metrics such as MAE, RMSE, and MdAE.

For DS-1, the dataset was partitioned into a training set comprising the initial 166 data points and a testing set consisting of the remaining 42 data points. The results obtained from DS-1 analysis revealed the superior performance of the proposed model in comparison to other models. This superiority is highlighted by smaller values of MAE (1.0388), RMSE (1.2383), and MdAE (1.0895), as presented in Table 1. Additionally, the visual representation in Figures 2 and 3 further emphasizes the performance advantage of the proposed model.

For DS-2, the dataset is partitioned into a training set, encompassing the initial 150 data points, and a testing set, comprising

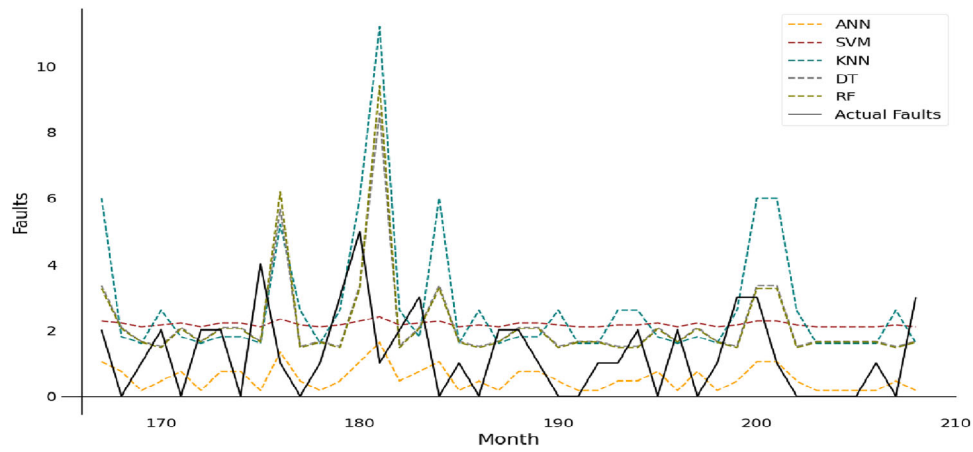


FIGURE 2 | Prediction plot of some machine learning models for DS-1.

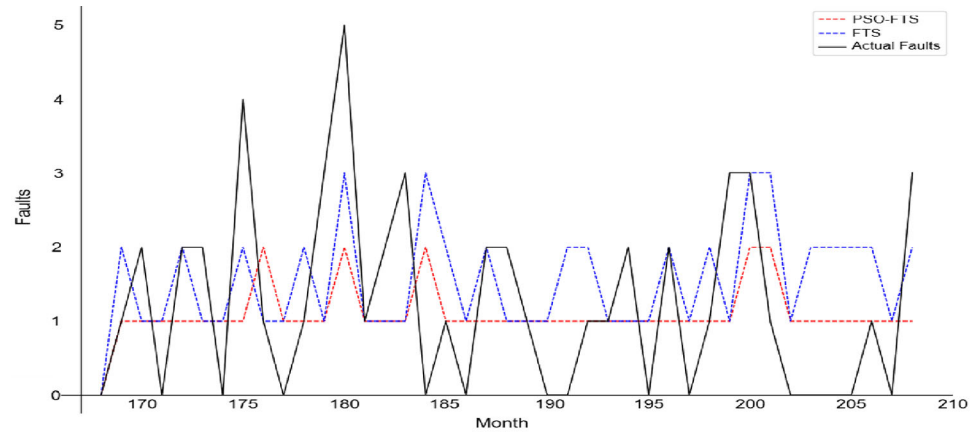


FIGURE 3 | Prediction plot of FTS and hybrid PSO-FTS for DS-1.

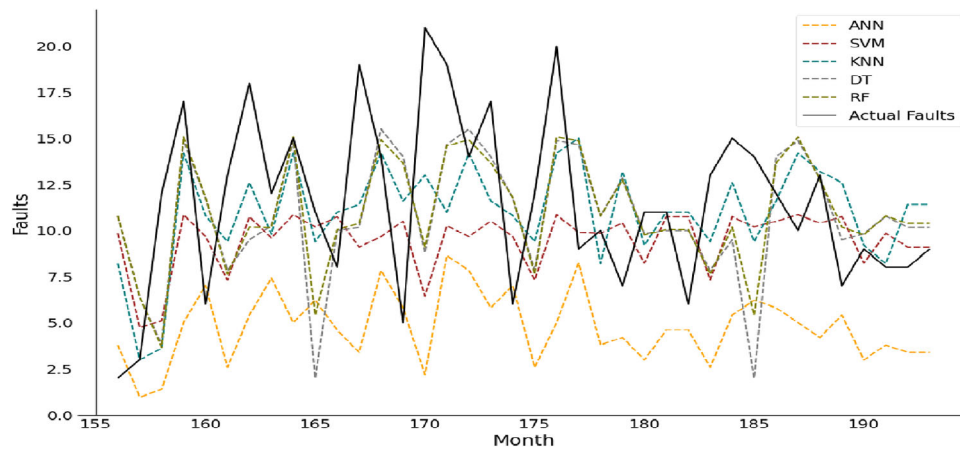


FIGURE 4 | Prediction plot of some machine learning models for DS-2.

the remaining 38 data points. Analysis reveals that the proposed model outshines alternative models in terms of performance. This superiority is manifested in lower values of MAE, RMSE, and MdAE, registering as 3.2453, 3.9739, and 3.2674, respectively (refer to Table 2). Additionally, the graphical representation in Figures 4 and 5 vividly illustrates the performance advantage of the proposed model. These findings firmly establish the

PSO-FTS model's efficacy and superiority in delivering precise fault predictions.

DS-3 is partitioned into a training set, consisting of the initial 154 data points, and a testing set, comprising the remaining 39 data points. A thorough analysis reveals the outstanding performance of the proposed model when compared to alternative models.

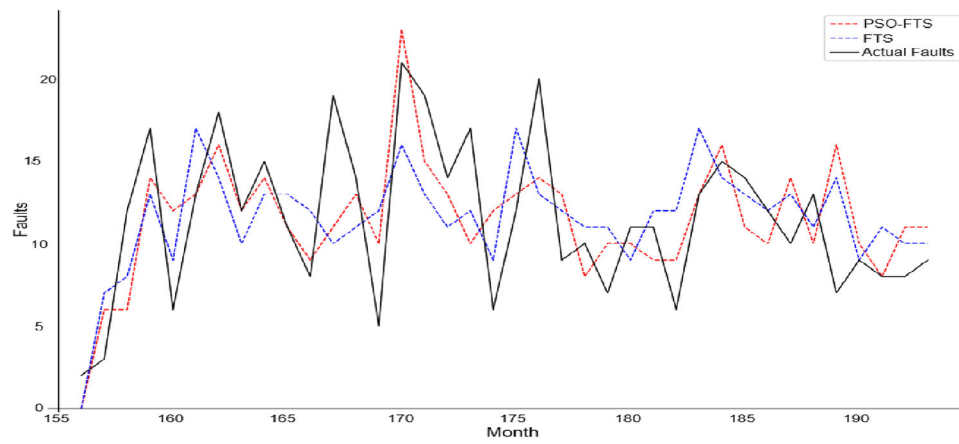


FIGURE 5 | Prediction plot of FTS and hybrid PSO-FTS for DS-2.

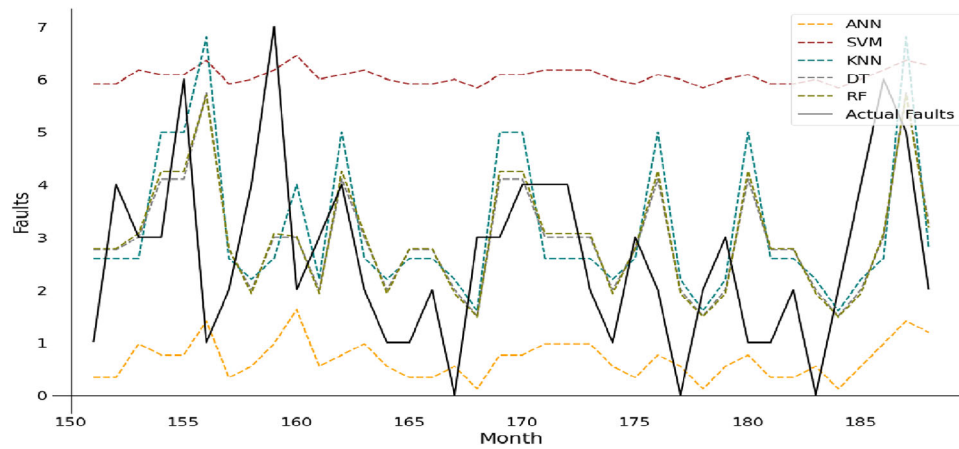


FIGURE 6 | Prediction plot of some machine learning models for DS-3.

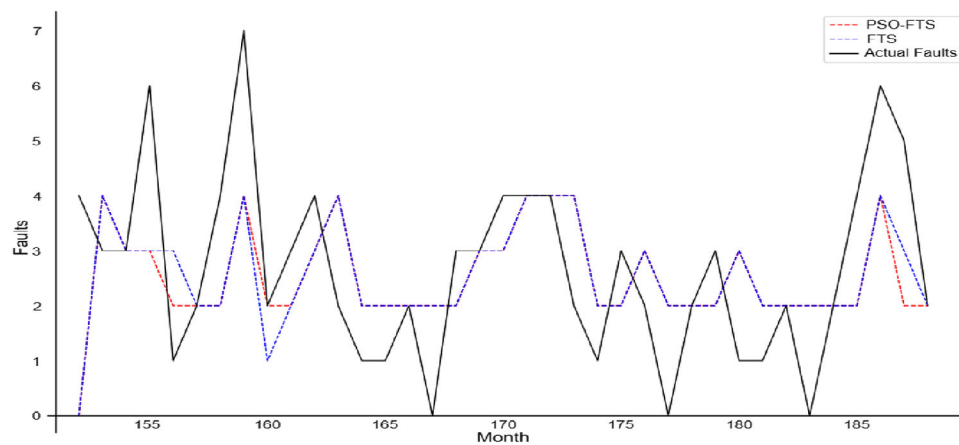


FIGURE 7 | Prediction plot of FTS and hybrid PSO-FTS for DS-3.

This superiority is discernible in the considerably lower values of MAE, RMSE, and MdAE, registering at 1.2818, 1.3974, and 2.2453, respectively (refer to Table 3). Additionally, the graphical representation in Figures 6 and 7 vividly illustrates the performance advantage of the proposed model. These findings robustly affirm the effectiveness and superiority of the PSO-FTS model in delivering precise fault predictions.

6 | Scalability and Limitations

The scalability of PSO-FTS is an important consideration, particularly when dealing with large datasets or real-time applications where quick predictions are required. While PSO is generally scalable and parallelizable, its integration with FTS introduces additional computational overhead. The optimization of multiple

ALGORITHM 3 | EvaluateUD Function.

```

1: function evaluateUD
   ( $m$ , particle,  $D_{\max}$ ,  $D_{\min}$ ,  $D_1$ ,  $D_2$ , series, faults, option = 1)
2:   if  $m < 1$  OR  $m > \text{series.size}$  then
3:     raise Exception ("value m is out of bounds")
4:   end if
5:    $n \leftarrow \text{particle.size} + 1$ 
6:    $x_{\min} \leftarrow \max(0, D_{\min} - D_1)$ 
7:    $x_{\max} \leftarrow D_{\max} + D_2$ 
8:   for  $id$  from 1 to  $n - 1$  do
9:      $U[id, 0] \leftarrow \text{particle}[id - 1]$ 
10:     $U[id, 1] \leftarrow \text{particle}[id]$ 
11:   end for
12:    $U[0, 0] \leftarrow x_{\min}$ 
13:    $U[0, 1] \leftarrow \text{particle}[0]$ 
14:    $U[n - 1, 0] \leftarrow \text{particle}[n - 2]$ 
15:    $U[n - 1, 1] \leftarrow x_{\max}$ 
16:   for  $idx$  from 0 to  $m - 1$  do
17:     for  $jdx$  from 0 to  $n - 1$  do
18:        $\text{Interval}[idx, 1] \leftarrow \text{faults}[idx]$ 
19:       if  $U[jdx, 0] \leq \text{faults}[idx] \leq U[jdx, 1]$  then
20:          $\text{Interval}[idx, 0] \leftarrow jdx$ 
21:         if  $(U[jdx, 1] - U[jdx, 0] == 0)$  OR  $(\text{faults}[idx] - U[jdx, 0] == 0)$  then
22:            $\text{Interval}[idx, 2] \leftarrow 1$ 
23:         else
24:            $\text{Interval}[idx, 2] \leftarrow \text{getMembershipValue}(\text{faults}[idx],$ 
              $U[jdx, 0], U[jdx, 1], \text{option})$ 
25:         end if
26:          $\text{Interval}[idx, 3] \leftarrow U[jdx, 0]$ 
27:          $\text{Interval}[idx, 4] \leftarrow U[jdx, 1]$ 
28:       end if
29:     end for
30:   end for
31:   for  $idx$  from 1 to  $m - 1$  do
32:      $LR[idx - 1, 0] \leftarrow \text{Interval}[idx - 1, 0]$ 
33:      $LR[idx - 1, 1] \leftarrow \text{Interval}[idx, 0]$ 
34:      $LR[idx - 1, 2] \leftarrow \text{Interval}[idx, 3]$ 
35:      $LR[idx - 1, 3] \leftarrow \text{Interval}[idx, 4]$ 
36:   end for
37:    $F \leftarrow \text{Forecast}[:, m, :], Ft \leftarrow \text{Forecast}[:, m, 2 : 4]$ 
38:    $F_{\text{next}} \leftarrow \text{round}(\text{Forecast}[m, 3])$ 
39:    $RMSE \leftarrow \sqrt{\text{sum}(\text{Forecast}[:, 5]) / (m - 1)}$ 
40:   return
      $n, x_{\min}, x_{\max}, U, \text{Interval}, LR, F, Ft, F_{\text{next}}, RMSE$ 
41: end Function
42: function evaluateUD
   ( $n, x_{\min}, x_{\max}, U, \text{Interval}, LR, F, Ft, F_{\text{next}}, RMSE,$ 
      $\text{Forecast}, m$ 
43:    $MAE \leftarrow \text{sum}(\text{Forecast}[:, 6]) / (m - 1)$ 
44:   return  $F, Ft, RMSE, U, F_{\text{next}}, \text{Forecast}[:, 5], MAE$ 
45: end Function

```

TABLE 1 | Comparison of results for DS-1.

Sl.No.	Model	MAE	RMSE	MdAE
1	ANN	1.1463	1.4813	0.8459
2	SVM	1.3271	1.5491	1.1868
3	KNN	1.8334	2.6068	1.6001
4	DT	1.4669	1.9651	1.4814
5	RF	1.4587	1.9564	1.4361
6	FTS	1.1178	1.2961	1.4011
7	PSO-FTS	1.0388	1.2383	1.0895

TABLE 2 | Comparison of results for DS-2.

Sl.No.	Model	MAE	RMSE	MdAE
1	ANN	6.5201	7.8203	5.5819
2	SVM	4.1913	5.2115	3.7781
3	KNN	4.1962	4.3932	4.5113
4	DT	4.4142	5.4568	1.4.0834
5	RF	4.1097	5.0209	4.0191
6	FTS	3.5871	4.1301	4.1875
7	PSO-FTS	3.2453	3.9739	3.2674

TABLE 3 | Comparison of results for DS-3.

Sl.No.	Model	MAE	RMSE	MdAE
1	ANN	3.1684	3.6569	3.0091
2	SVM	3.4621	3.7905	3.8302
3	KNN	1.6165	1.9971	1.4129
4	DT	1.4167	1.7339	1.0526
5	RF	1.4102	1.7258	1.1688
6	FTS	1.3111	1.5892	2.6875
7	PSO-FTS	1.2818	1.3974	2.2453

FTS parameters, such as interval length and fuzzy relationships, increases the computational complexity, especially for large-scale datasets. Furthermore, maintaining multiple particles and their historical best positions in PSO can lead to high memory usage, which may become a constraint when processing extensive time series data. The iterative nature of PSO also impacts real-time applications, as multiple iterations are required to reach an optimal solution, potentially leading to increased inference time.

To address these challenges, several strategies can be considered. Parallel and distributed implementations of PSO can significantly reduce computation time by leveraging multi-core processors or cloud-based frameworks. Adaptive PSO mechanisms, where optimization is triggered only when performance degradation is detected, can help minimize unnecessary computations. Additionally, hybrid approaches that integrate PSO with more computationally efficient methods, such as genetic algorithms or reinforcement learning, can provide a balance between accuracy

and efficiency. These techniques can enhance the scalability of PSO-FTS, making it more suitable for large-scale and real-time applications.

However, several limitations must be acknowledged. The model's performance is influenced by the selection of hyperparameters, and alternative tuning strategies could further refine its accuracy. Additionally, while the approach has been validated on specific datasets, its generalization to other domains requires further exploration. The assumption of stationarity in the dataset may not always hold in real-world applications, where abrupt changes and non-stationary behaviors occur. Finally, while PSO was chosen for its efficiency and simplicity, future studies could explore the application of other metaheuristic algorithms such as GA or DE to compare their effectiveness in optimizing FTS forecasting.

7 | Conclusion

This study introduced a hybrid forecasting model that integrates PSO with FTS for software fault prediction. The primary objective was to enhance the predictive accuracy of FTS by optimizing the length of interval partitions using PSO. Experimental evaluations were conducted on three real-world software fault datasets, demonstrating the effectiveness of the proposed PSO-FTS model.

The performance of PSO-FTS was compared with traditional FTS as well as several machine learning models, including ANN, SVM, KNN, DT, and RF. The results showed that PSO-FTS consistently outperformed these methods across multiple evaluation metrics, including MAE, RMSE, and MdAE. Specifically, for DS-1, the proposed model achieved an MAE of 1.0388, RMSE of 1.2383, and MdAE of 1.0895, outperforming ANN, SVM, and other models. Similarly, for DS-2, PSO-FTS achieved an MAE of 3.2453, RMSE of 3.9739, and MdAE of 3.2674, reinforcing its superior predictive accuracy. For DS-3, the proposed model attained an MAE of 1.2818, RMSE of 1.3974, and MdAE of 2.2453, further confirming its robustness.

By leveraging the strengths of PSO to optimize the interval lengths in FTS forecasting, the model improves predictive accuracy and adaptability across different datasets. The results indicate that PSO-FTS is a promising approach for handling uncertainty in time series analysis. Future work should focus on refining the model's robustness by incorporating adaptive mechanisms for hyperparameter selection, expanding its applicability to diverse datasets, and exploring hybrid optimization techniques to enhance its forecasting capabilities. By addressing these challenges, PSO-FTS can be further improved to provide more accurate and reliable predictions in various real-world scenarios.

Data Availability Statement

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

References

1. U. Samal, "Software Reliability Growth Model Considering Imperfect Debugging and Fault Removal Efficiency," *Quality and Reliability Engineering International* (2024), <https://doi.org/10.1002/qre.3716>.

2. S. Gupta, A. Mishra, and M. Chawla, *Analysis and Recommendation of Common Fault and Failure in Software Development Systems* (IEEE, 2016), 1730–1734.
3. W. E. Wong, X. Li, and P. A. Laplante, "Be More Familiar With Our Enemies and Pave the Way Forward: A Review of the Roles Bugs Played in Software Failures," *Journal of Systems and Software* 133 (2017): 68–94, <https://doi.org/10.1016/j.jss.2017.06.069>.
4. U. Samal and A. Kumar, "Redefining Software Reliability Modeling: Embracing Fault-Dependency, Imperfect Removal, and Maximum Fault Considerations," *Quality Engineering* 36 (2023): 500–509, <https://doi.org/10.1080/08982112.2023.2241067>.
5. Y. S. Huang, K. C. Chiu, and W. M. Chen, "A Software Reliability Growth Model for Imperfect Debugging," *Journal of Systems and Software* 188 (2022): 111267, <https://doi.org/10.1016/j.jss.2022.111267>.
6. V. Nagaraju, L. Fiondella, and T. Wandji, "A Heterogeneous Single Changepoint Software Reliability Growth Model Framework," *Software Testing, Verification and Reliability* 29, no. 8 (2019): e1717, <https://doi.org/10.1002/stvr.1717>.
7. M. Asraful Haque and N. Ahmad, "A Logistic Growth Model for Software Reliability Estimation Considering Uncertain Factors," *International Journal of Reliability, Quality and Safety Engineering* 28, no. 05 (2021): 2150032, <https://doi.org/10.1142/S0218539321500327>.
8. U. Samal and A. Kumar, "A Software Reliability Model Incorporating Fault Removal Efficiency and Its Release Policy," *Computational Statistics* 39 (2023): 3137–3155, <https://doi.org/10.1007/s00180-023-01430-9>.
9. U. Samal and A. Kumar, "Empowering Software Reliability: Leveraging Efficient Fault Detection and Removal Efficiency," *Quality Engineering* 37, no. 1 (2025): 118–129, <https://doi.org/10.1080/08982112.2024.2358889>.
10. U. Samal, S. Kushwaha, and A. Kumar, "A Testing-Effort Based SRGM Incorporating Imperfect Debugging and Change Point," *Reliability: Theory & Applications* 18, no. 1 (72) (2023): 86–93.
11. N. Fenton, M. Neil, and D. Marquez, "Using Bayesian Networks to Predict Software Defects and Reliability," *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability* 222, no. 4 (2008): 701–712, <https://doi.org/10.1243/1748006XJRR161>.
12. X. Liu and N. Xie, "Grey-Based Approach for Estimating Software Reliability Under Nonhomogeneous Poisson Process," *Journal of Systems Engineering and Electronics* 33, no. 2 (2022): 360–369, <https://doi.org/10.23919/JSEE.2022.000038>.
13. R. K. Behera, S. Shukla, S. K. Rath, and S. Misra, *Software Reliability Assessment Using Machine Learning Technique* (Springer, 2018), 403–411.
14. U. Samal and A. Kumar, "Enhancing Software Reliability Forecasting Through a Hybrid ARIMA-ANN Model," *Arabian Journal for Science and Engineering* 49 (2023): 7571–7584, <https://doi.org/10.1007/s13369-023-08486-1>.
15. K. Kumaresan and P. Ganeshkumar, "Software Reliability Prediction Model With Realistic Assumption Using Time Series (S) ARIMA Model," *Journal of Ambient Intelligence and Humanized Computing* 11 (2020): 5561–5568, <https://doi.org/10.1007/s12652-020-01912-4>.
16. K. K. Raghuvanshi, A. Agarwal, K. Jain, and V. Singh, "A Generalized Prediction Model for Improving Software Reliability Using Time-Series Modelling," *International Journal of System Assurance Engineering and Management* 13, no. 3 (2022): 1309–1320, <https://doi.org/10.1007/s13198-021-01449-5>.
17. U. Samal and A. Kumar, "A Neural Network Approach for Software Reliability Prediction," *International Journal of Reliability, Quality and Safety Engineering* 31 (2024): 2450009, <https://doi.org/10.1142/S0218539324500098>.
18. S. Singh, "A Simple Method of Forecasting Based on Fuzzy Time Series," *Applied Mathematics and Computation* 186, no. 1 (2007): 330–339.

19. S. H. Cheng, S. M. Chen, and W. S. Jian, "Fuzzy Time Series Forecasting Based on Fuzzy Logical Relationships and Similarity Measures," *Information Sciences* 327 (2016): 272–287.
20. S. Zeng, S. M. Chen, and M. O. Teng, "Fuzzy Forecasting Based on Linear Combinations of Independent Variables, Subtractive Clustering Algorithm and Artificial Bee Colony Algorithm," *Information Sciences* 484 (2019): 350–366.
21. Q. Song and B. S. Chissom, "Forecasting Enrollments With Fuzzy Time Series—Part I," *Fuzzy Sets and Systems* 54, no. 1 (1993): 1–9.
22. Q. Song and B. S. Chissom, "Forecasting Enrollments With Fuzzy Time Series—Part II," *Fuzzy Sets and Systems* 62, no. 1 (1994): 1–8.
23. C. A. Severiano, P. C. Silva, H. J. Sadaei, and F. G. Guimarães, *Very Short-Term Solar Forecasting Using Fuzzy Time Series* (IEEE, 2017), 1–6.
24. S. M. Chen, X. Y. Zou, and G. C. Gunawan, "Fuzzy Time Series Forecasting Based on Proportions of Intervals and Particle Swarm Optimization Techniques," *Information Sciences* 500 (2019): 127–139.
25. M. Bose and K. Mali, "Designing Fuzzy Time Series Forecasting Models: A Survey," *International Journal of Approximate Reasoning* 111 (2019): 78–99, <https://doi.org/10.1016/j.ijar.2019.05.002>.
26. K. Y. Cai, C. Y. Wen, and M. L. Zhang, "A Novel Approach to Software Reliability Modeling," *Microelectronics Reliability* 33, no. 15 (1993): 2265–2267.
27. W. Zhang, S. Zhang, S. Zhang, D. Yu, and N. Huang, "A Novel Method Based on FTS With Both GA-FCM and Multifactor BPNN for Stock Forecasting," *Soft Computing* 23 (2019): 6979–6994, <https://doi.org/10.1007/s00500-018-3335-2>.
28. M. Pant and S. Kumar, "Fuzzy Time Series Forecasting Based on Hesitant Fuzzy Sets, Particle Swarm Optimization and Support Vector Machine-Based Hybrid Method," *Granular Computing* 7 (2022): 1–19, <https://doi.org/10.1007/s41066-021-00300-3>.
29. I. H. Kuo, S. J. Horng, T. W. Kao, T. L. Lin, C. L. Lee, and Y. Pan, "An Improved Method for Forecasting Enrollments Based on Fuzzy Time Series and Particle Swarm Optimization," *Expert Systems With Applications* 36, no. 3 (2009): 6108–6117.
30. A. L. Goel and K. Okumoto, "Time-Dependent Error-Detection Rate Model for Software Reliability and Other Performance Measures," *IEEE Transactions on Reliability* R-28, no. 3 (1979): 206–211, <https://doi.org/10.1109/TR.1979.5220566>.
31. S. Yamada, M. Ohba, and S. Osaki, "S-Shaped Software Reliability Growth Models and Their Applications," *IEEE Transactions on Reliability* 33, no. 4 (1984): 289–292, <https://doi.org/10.1109/TR.1984.5221826>.
32. K. Sahu and R. Srivastava, "Revisiting Software Reliability," in *Data Management, Analytics and Innovation: Proceedings of ICDMAI 2018, Volume 1* (Springer, 2019): 221–235.
33. M. A. Haque and N. Ahmad, "Key Issues in Software Reliability Growth Models," *Recent Advances in Computer Science and Communications (Formerly: Recent Patents on Computer Science)* 15, no. 5 (2022): 741–747, <https://doi.org/10.2174/2666255813999201012182821>.
34. R. M. Pattanayak, S. Panigrahi, and H. Behera, "High-Order Fuzzy Time Series Forecasting by Using Membership Values Along With Data and Support Vector Machine," *Arabian Journal for Science and Engineering* 45, no. 12 (2020): 10311–10325, <https://doi.org/10.1007/s13369-020-04721-1>.
35. S. Panigrahi and H. S. Behera, "A Study on Leading Machine Learning Techniques for High Order Fuzzy Time Series Forecasting," *Engineering Applications of Artificial Intelligence* 87 (2020): 103245, <https://doi.org/10.1016/j.engappai.2019.103245>.
36. S. Chatterjee, S. Nigam, J. Singh, and L. Upadhyaya, "Application of Fuzzy Time Series in Prediction of Time Between Failures & Faults in Software Reliability Assessment," *Fuzzy Information and Engineering* 3 (2011): 293–309, <https://doi.org/10.1007/s12543-011-0084-7>.
37. G. Junhong, Y. Xiaozong, and L. Hongwei, "Software Reliability Nonlinear Modeling and Its Fuzzy Evaluation," (2005).
38. S. Aljahdali and A. F. Sheta, *Predicting the Reliability of Software Systems Using Fuzzy Logic* (IEEE, 2011), 36–40.
39. L. Zhao, J. P. Zhang, J. Yang, and Y. Chu, *Software Reliability Growth Model Based on Fuzzy Wavelet Neural Network* (IEEE, 2010), V1–664.
40. S. Chatterjee, B. Chaudhuri, and C. Bhar, "Optimal Release Time Determination Via Fuzzy Goal Programming Approach for SDE-Based Software Reliability Growth Model," *Soft Computing* 25 (2021): 3545–3564, <https://doi.org/10.1007/s00500-020-05385-7>.
41. A. Kumar, "A Neuro-Fuzzy Hybridized Approach for Software Reliability Prediction," *JUCS: Journal of Universal Computer Science* 28, no. 7 (2022): 708–732, <https://doi.org/10.3897/jucs.80537>.
42. L. A. Zadeh, "Fuzzy Sets," *Information and Control* 8, no. 3 (1965): 338–353, [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X).
43. U. Samal, S. Kushwaha, G. Nain, S. Singh, S. Usmani, and A. Kumar, *Necessity of Fuzzy Logic: Trends in Software Reliability Assessment* (Elsevier, 2025), 275–285.
44. N. Talpur, S. J. Abdulkadir, H. Alhussian, M. H. Hasan, N. Aziz, and A. Bamhdi, "Deep Neuro-Fuzzy System Application Trends, Challenges, and Future Perspectives: A Systematic Survey," *Artificial Intelligence Review* 56, no. 2 (2023): 865–913, <https://doi.org/10.1007/s10462-022-10188-3>.
45. Y. Zhang, S. Wang, and G. Ji, "A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications," *Mathematical Problems in Engineering* 2015, no. 1 (2015): 931256, <https://doi.org/10.1155/2015/931256>.
46. S. Panda and N. P. Padhy, "Comparison of Particle Swarm Optimization and Genetic Algorithm for FACTS-Based Controller Design," *Applied Soft Computing* 8, no. 4 (2008): 1418–1427, <https://doi.org/10.1016/j.asoc.2007.10.009>.
47. V. Kachitvichyanukul, "Comparison of Three Evolutionary Algorithms: GA, PSO, and DE," *Industrial Engineering and Management Systems* 11, no. 3 (2012): 215–223, <https://doi.org/10.7232/iems.2012.11.3.215>.
48. M. Clerc and J. Kennedy, "The Particle Swarm-Explosion, Stability, and Convergence in a Multidimensional Complex Space," *IEEE Transactions on Evolutionary Computation* 6, no. 1 (2002): 58–73, <https://doi.org/10.1109/4235.985692>.
49. C. Y. Wu and C. Y. Huang, "A Study of Incorporation of Deep Learning Into Software Reliability Modeling and Assessment," *IEEE Transactions on Reliability* 70, no. 4 (2021): 1621–1640, <https://doi.org/10.1109/TR.2021.3105531>.

TABLE A1 | DS-1.

10991638, 2025, 5, Downloaded from <https://onlinelibrary.wiley.com/doi/10.1002/qe.3759> by Nait Institution Of Tech, Hampanu, Wiley Online Library on [25/11/2025]. See the Terms and Conditions (<https://onlinelibrary.wiley.com/terms-and-conditions>) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE A3 | DS-3.

Month		Fault	Month	Fault	Month	Fault	Month	Fault	Month	Fault	Month	Fault	Month	Fault	Month	Fault	Month	Fault	Month	Fault
1	0	21	13	41	9	61	7	81	11	101	9	121	3	141	12	161	13	181	11	
2	1	22	10	42	19	62	7	82	6	102	10	122	5	142	11	162	18	182	6	
3	1	23	8	43	6	63	14	83	7	103	11	123	3	143	10	163	12	183	13	
4	0	24	11	44	2	64	19	84	12	104	6	124	5	144	25	164	15	184	15	
5	2	25	11	45	12	65	25	85	17	105	9	125	4	145	2	165	11	185	14	
6	2	26	15	46	12	66	20	86	8	106	2	126	3	146	24	166	8	186	12	
7	2	27	2	47	14	67	10	87	26	107	0	127	5	147	13	167	19	187	10	
8	1	28	5	48	12	68	11	88	12	108	0	128	5	148	5	168	14	188	13	
9	18	29	17	49	8	69	11	89	5	109	0	129	4	149	10	169	5	189	7	
10	6	30	5	50	18	70	12	90	10	110	6	130	2	150	5	170	21	190	9	
11	2	31	9	51	6	71	46	91	14	111	2	131	6	151	2	171	19	191	8	
12	3	32	7	52	16	72	13	92	17	112	2	132	72	152	7	172	14	192	8	
13	2	33	10	53	7	73	9	93	16	113	2	133	19	153	10	173	17	193	9	
14	6	34	11	54	16	74	6	94	7	114	53	134	7	154	5	174				
15	4	35	9	55	4	75	16	95	7	115	49	135	8	155	9	175	12			
16	6	36	14	56	10	76	10	96	3	116	24	136	4	156	2	176	20			
17	6	37	8	57	22	77	31	97	6	117	93	137	22	157	3	177	9			
18	2	38	4	58	5	78	17	98	8	118	9	138	12	158	128	178	10			
19	3	39	12	59	25	79	18	99	10	119	23	139	9	159	17	179	7			
20	0	40	8	60	16	80	16	100	9	120	10	140	7	160	6	180	11			